Task 2 - Part D

Zhuoyan Jiang

2026-10-17

Part D: Application to the Faithful Data

Overview

This part applies the Gibbs sampler of Part A to the `eruptions` variable of R's built-in `faithful` data set, to see whether a two-component Gaussian mixture describes these data well.

- **Sections 1 and 2** give the sampler, the standardized data, the prior and the four chains.
- **Section 3** checks the chains for label switching and relabels the draws.
- **Section 4** assesses the MCMC performance and reports the posterior summaries.
- **Sections 5 to 7** compare the fitted mixture with the data, look at the uncertainty of the allocations, and check the adequacy of the model.

---

1. Gibbs sampler from Part A

The sampler is taken from Part A so that this file can be knitted on its own.

```
gibbs_sampler <- function(y, n_iter = 12000,
                          m0 = 0, s0 = 1,
                          a = 1, b = 1,
                          alpha0 = 2, beta0 = 1,
                          mu1_init = -1,
                          mu2_init = 1,
                          pi_init = 0.5,
                          sigma2_init = 1) {

  n <- length(y)

  # Store posterior draws
  mu1 <- numeric(n_iter)
  mu2 <- numeric(n_iter)
  pi <- numeric(n_iter)
  sigma2 <- numeric(n_iter)

  # Store latent allocations
  Z <- matrix(NA_integer_, nrow = n_iter, ncol = n)

  # Initial parameter values
  mu1[1] <- mu1_init
  mu2[1] <- mu2_init
  pi[1] <- pi_init
  sigma2[1] <- sigma2_init
```

```r
  # Initial allocations
  Z[1, ] <- ifelse(y < (mu1_init + mu2_init) / 2, 1L, 2L)

  for (t in 2:n_iter) {

    # Update Z_i: log densities of the two components
    log_f1 <- dnorm(y, mean = mu1[t - 1], sd = sqrt(sigma2[t - 1]), log = TRUE)
    log_f2 <- dnorm(y, mean = mu2[t - 1], sd = sqrt(sigma2[t - 1]), log = TRUE)

    # P(Z_i = 1 | rest)
    w <- plogis(log(pi[t - 1]) + log_f1 - log1p(-pi[t - 1]) - log_f2)

    # Component 1 if the uniform draw is below w, otherwise component 2
    Z[t, ] <- ifelse(runif(n) < w, 1L, 2L)
    z <- Z[t, ]
    n1 <- sum(z == 1)
    n2 <- sum(z == 2)

    # Update mu1
    v1 <- 1 / (n1 / sigma2[t - 1] + 1 / s0^2)
    m1 <- v1 * (sum(y[z == 1]) / sigma2[t - 1] + m0 / s0^2)
    mu1[t] <- rnorm(1, mean = m1, sd = sqrt(v1))

    # Update mu2
    v2 <- 1 / (n2 / sigma2[t - 1] + 1 / s0^2)
    m2 <- v2 * (sum(y[z == 2]) / sigma2[t - 1] + m0 / s0^2)
    mu2[t] <- rnorm(1, mean = m2, sd = sqrt(v2))

    # Update pi
    pi[t] <- rbeta(1, shape1 = a + n1, shape2 = b + n2)

    # Update sigma2
    mu_z <- ifelse(z == 1, mu1[t], mu2[t])
    alpha_n <- alpha0 + n / 2
    beta_n <- beta0 + 0.5 * sum((y - mu_z)^2)
    sigma2[t] <- 1 / rgamma(1, shape = alpha_n, rate = beta_n)
  }

  return(list(draws = data.frame(mu1, mu2, pi, sigma2), Z = Z))
}
```

---

2. Data, prior and sampler runs

The data are standardized, so that the prior of Part A can be used on a common scale. All results below are transformed back to minutes.

```r
data(faithful)
y <- faithful$eruptions
n <- length(y)

# Standardize; the original mean and sd are kept to transform results back
y_mean <- mean(y)
```

```r
y_sd <- sd(y)
ys <- (y - y_mean) / y_sd

cat("n =", n)
```

```
## n = 272
```

```r
cat("Mean and sd of eruptions (minutes):", round(c(y_mean, y_sd), 3))
```

```
## Mean and sd of eruptions (minutes): 3.488 1.141
```

The prior is set on the standardized scale: $\mu_1, \mu_2 \sim N(0, 5^2)$, $\pi \sim \text{Beta}(1, 1)$ and $\sigma^2 \sim \text{InvGamma}(2, 1)$. These priors are weakly informative: the standardized data lie roughly between -1.7 and 1.4, so $N(0, 5^2)$ for the means covers the data range several times.

```r
m0 <- 0
s0 <- 5
a <- 1
b <- 1
alpha0 <- 2
beta0 <- 1
```

Four chains are run from different starting values. The starting means are placed on both sides of zero and are swapped between chains, so that the chains start in both labellings. The starting values of π and σ^2 also differ.

```r
n_iter <- 12000
burnin <- 2000

inits <- data.frame(
  mu1 = c(-1, 1, -2, 2),
  mu2 = c(1, -1, 2, -2),
  pi = c(0.5, 0.3, 0.7, 0.5),
  sigma2 = c(1, 0.5, 2, 1)
)

set.seed(1)
chains <- lapply(1:4, function(j) {
  gibbs_sampler(ys, n_iter = n_iter, m0 = m0, s0 = s0, a = a, b = b,
                alpha0 = alpha0, beta0 = beta0,
                mu1_init = inits$mu1[j], mu2_init = inits$mu2[j],
                pi_init = inits$pi[j], sigma2_init = inits$sigma2[j])
})
```

---

3. Label switching in the output

Before any post-processing, each chain is checked for label switching. A swap means that $\mu_1 > \mu_2$ at an iteration.

```r
raw_summary <- t(sapply(chains, function(ch) {
  d <- ch$draws[-(1:burnin), ]
  swap <- d$mu1 > d$mu2
  c(mu1 = mean(d$mu1), mu2 = mean(d$mu2), pi = mean(d$pi),
    share_mu1_above_mu2 = mean(swap),
    label_changes = sum(diff(swap) != 0))
```

```
}))
rownames(raw_summary) <- paste("chain", 1:4)
round(raw_summary, 3)
```

```
##            mu1    mu2    pi share_mu1_above_mu2 label_changes
## chain 1 -1.262  0.709 0.361                   0             0
## chain 2  0.709 -1.262 0.639                   1             0
## chain 3 -1.261  0.710 0.361                   0             0
## chain 4  0.709 -1.261 0.639                   1             0
```

Within each chain the labelling never changes (0 label changes). Chains 2 and 4 are in the swapped labelling compared with chains 1 and 3 ($\mu_1 > \mu_2$ and $\pi \approx 0.64$ instead of 0.36), because they started from swapped initial values. Both labellings describe the same mixture: means of -1.26 and 0.71 on the standardized scale, with weights 0.36 and 0.64. Pooling the chains without relabelling would mix the two components, as discussed in Part C. Because the two groups are well separated, no chain switches labelling.

The labels are arbitrary, so the draws are relabelled such that component "low" is always the one with the smaller mean. This puts all chains on the same labelling. In every iteration with $\mu_1 > \mu_2$:

- the two means are swapped, so that $\mu_{\text{low}} = \mu_2$ and $\mu_{\text{high}} = \mu_1$;
- the weight of the low component is $1 - \pi$ instead of π, because π is the weight of component 1, which is now the high component;
- the allocations Z_i are switched between 1 and 2, so that they refer to the new labels.

Iterations with $\mu_1 < \mu_2$ are left unchanged. After this step, all four chains describe the same two components.

```
relabel <- function(ch, burnin) {
  d <- ch$draws[-(1:burnin), ]
  Z <- ch$Z[-(1:burnin), ]
  swap <- d$mu1 > d$mu2
  Z[swap, ] <- 3L - Z[swap, ]
  list(
    draws = data.frame(mu_low = pmin(d$mu1, d$mu2),
                       mu_high = pmax(d$mu1, d$mu2),
                       pi_low = ifelse(swap, 1 - d$pi, d$pi),
                       sigma2 = d$sigma2),
    # Posterior probability that observation i belongs to the high component
    p_high = colMeans(Z == 2)
  )
}

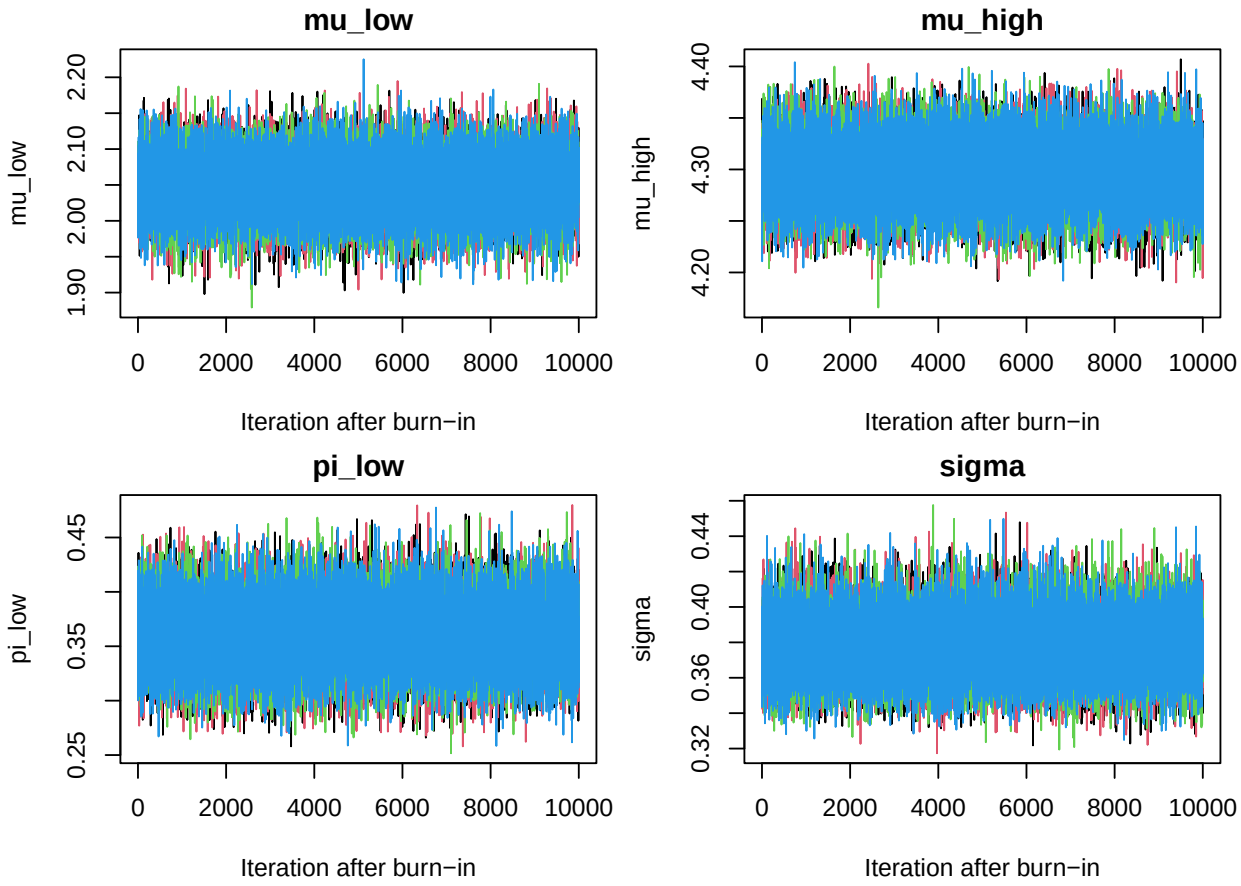
# Transform back to the original scale (minutes)
to_orig <- function(d) {
  data.frame(mu_low = d$mu_low * y_sd + y_mean,
             mu_high = d$mu_high * y_sd + y_mean,
             pi_low = d$pi_low,
             sigma = sqrt(d$sigma2) * y_sd)
}

rel <- lapply(chains, relabel, burnin = burnin)
ord <- lapply(rel, function(r) to_orig(r$draws))
pooled <- do.call(rbind, ord)
```

4. MCMC performance and posterior summaries

Trace plots of the relabelled chains, original scale:

```
par(mfrow = c(2, 2), mar = c(4, 4, 2, 1))
for (p in c("mu_low", "mu_high", "pi_low", "sigma")) {
  vals <- sapply(ord, function(d) d[[p]])
  matplot(vals, type = "l", lty = 1, col = 1:4, main = p,
          xlab = "Iteration after burn-in", ylab = p)
}
```



The potential scale reduction factor $\hat{R}$ (Gelman–Rubin, computed on the four relabelled chains) and the effective sample size (ESS, summed over the chains, from the autocorrelation of each chain truncated at the first negative value) are computed directly.

```
rhat <- function(x) {
  # x: matrix with one column per chain
  m <- ncol(x)
  k <- nrow(x)
  W <- mean(apply(x, 2, var))
  B <- k * var(colMeans(x))
  sqrt(((k - 1) / k * W + B / k) / W)
}

ess <- function(x) {
  k <- length(x)
  rho <- acf(x, lag.max = 1000, plot = FALSE)$acf[-1]
```

```
  cut <- which(rho < 0)[1] - 1
  if (is.na(cut)) cut <- length(rho)
  k / (1 + 2 * sum(rho[seq_len(cut)]))
}

pars <- c("mu_low", "mu_high", "pi_low", "sigma")
summ <- t(sapply(pars, function(p) {
  x <- sapply(ord, function(d) d[[p]])
  c(mean = mean(x), sd = sd(x),
    q2.5 = unname(quantile(x, 0.025)), q97.5 = unname(quantile(x, 0.975)),
    Rhat = rhat(x), ESS = sum(apply(x, 2, ess)))
}))
summ_out <- round(summ, 3)
summ_out[, "ESS"] <- round(summ[, "ESS"])
knitr::kable(summ_out)
```

	mean	sd	q2.5	q97.5	Rhat	ESS
mu_low	2.048	0.039	1.972	2.124	1	36086
mu_high	4.297	0.029	4.241	4.353	1	37964
pi_low	0.361	0.029	0.305	0.419	1	38646
sigma	0.377	0.016	0.346	0.411	1	36967

The relabelled chains agree closely, and the traces overlap without any drift. $\hat{R}$ is 1.00 for all four parameters. The effective sample sizes are above 36,000 out of 40,000 stored draws, so the draws are almost uncorrelated.

The table also gives the posterior means, standard deviations and 95% credible intervals of the relabelled parameters, in minutes. Component "low" has the smaller mean, "high" the larger one, and σ is the common standard deviation of the two components. The lower component has mean 2.05 minutes (95% interval [1.97, 2.12]) and weight 0.36 ([0.305, 0.419]). The upper component has mean 4.30 minutes ([4.24, 4.35]) and weight 0.64. The common standard deviation is 0.38 minutes ([0.35, 0.41]).

---

5. Fitted mixture compared with the data

The fitted density is computed for every (thinned) posterior draw as $\pi f_{\text{low}} + (1-\pi) f_{\text{high}}$. This density does not depend on the labelling. The plot shows its posterior mean with pointwise 95% bands, together with the histogram and a kernel density estimate of the data.

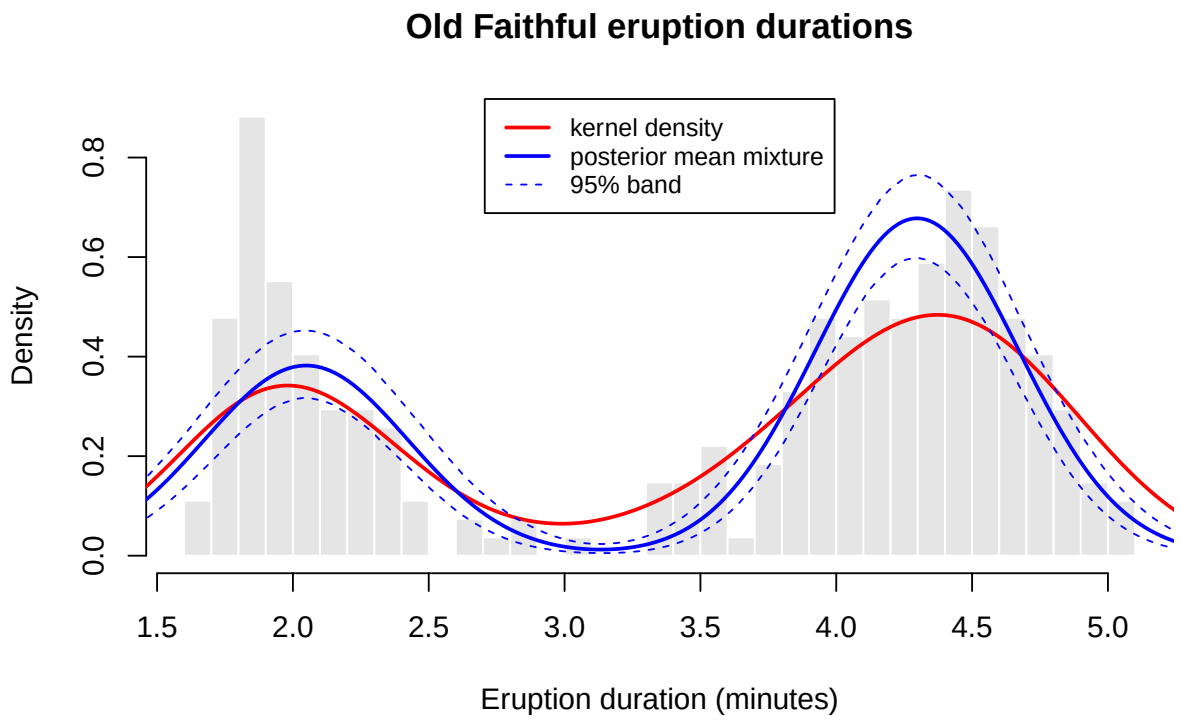
```
grid <- seq(1, 6, length.out = 300)
idx <- seq(1, nrow(pooled), by = 20)

dens <- sapply(idx, function(t) {
  pooled$pi_low[t] * dnorm(grid, pooled$mu_low[t], pooled$sigma[t]) +
    (1 - pooled$pi_low[t]) * dnorm(grid, pooled$mu_high[t], pooled$sigma[t])
})
dens_mean <- rowMeans(dens)
dens_lo <- apply(dens, 1, quantile, probs = 0.025)
dens_hi <- apply(dens, 1, quantile, probs = 0.975)

hist(y, breaks = 30, probability = TRUE, col = "gray90", border = "white",
     main = "Old Faithful eruption durations", xlab = "Eruption duration (minutes)")
lines(density(y), col = "red", lwd = 2)
lines(grid, dens_mean, col = "blue", lwd = 2)
```

```r
lines(grid, dens_lo, col = "blue", lty = 2)
lines(grid, dens_hi, col = "blue", lty = 2)

legend("top",
       c("kernel density", "posterior mean mixture", "95% band"),
       col = c("red", "blue", "blue"), lty = c(1, 1, 2),
       lwd = c(2, 2, 1), bg = "white", cex = 0.8)
```



The posterior mean mixture has its two peaks at about 2.0 and 4.3 minutes, in line with the two modes of the histogram, and the density between them is close to zero. The kernel density estimate has lower peaks and a higher valley than the fitted mixture. The 95% band is narrow, but it only reflects the uncertainty in the parameters, not how well the model fits. The fit is not perfect: the highest histogram bar of the short eruptions (near 1.8 minutes) is well above the fitted density, the mixture puts some density below the shortest observed duration (about 1.6 minutes), and it lies below the histogram on the left side of the long group, between about 3.3 and 3.6 minutes.

---

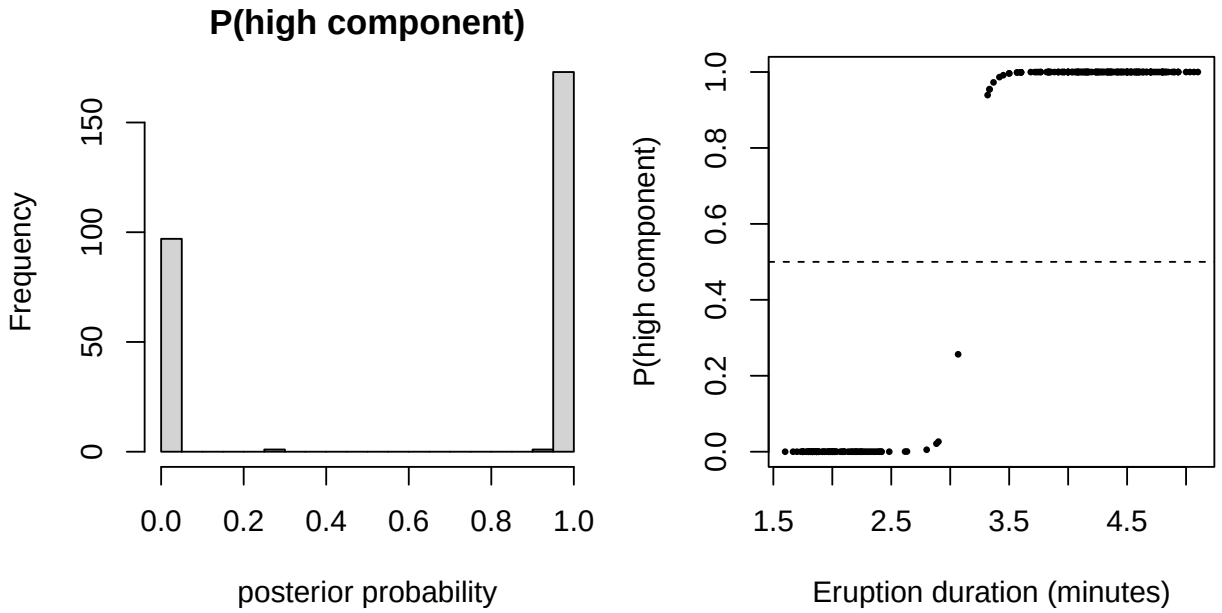
6. Uncertainty about the allocation

The posterior probability that observation i belongs to the high component is the share of draws that allocate it to that component, after relabelling.

```r
p_high <- rowMeans(sapply(rel, function(r) r$p_high))

par(mfrow = c(1, 2), mar = c(4, 4, 2, 1))
hist(p_high, breaks = 20, main = "P(high component)", xlab = "posterior probability")
plot(y, p_high, pch = 16, cex = 0.5, xlab = "Eruption duration (minutes)",
     ylab = "P(high component)")
```

```
abline(h = 0.5, lty = 2)
```




```
cat("Observations with P(high) > 0.5:", sum(p_high > 0.5))
```

```
## Observations with P(high) > 0.5: 174
```

```
cat("Observations with 0.05 < P(high) < 0.95:", sum(p_high > 0.05 & p_high < 0.95))
```

```
## Observations with 0.05 < P(high) < 0.95: 2
```

```
cat("Range of eruption durations with 0.05 < P(high) < 0.95:",
    round(range(y[p_high > 0.05 & p_high < 0.95]), 2))
```

```
## Range of eruption durations with 0.05 < P(high) < 0.95: 3.07 3.32
```

174 of the 272 observations have a posterior probability above 0.5 of belonging to the upper component, which agrees with the estimated weight of 0.64. Only 2 observations have a probability between 0.05 and 0.95 (the two thresholds are an arbitrary choice). Their durations are 3.07 and 3.32 minutes, in the gap between the two groups. The uncertainty about the allocation is therefore limited to the boundary between the components.

---

7. Is the two-component Gaussian mixture adequate?

A posterior predictive check is used. Data sets of the same size are simulated from the fitted model (parameters drawn from the posterior, then allocations and observations simulated), and the sample skewness and excess kurtosis are compared with the observed values. The posterior predictive p-value is the share of replicated values at least as large as the observed one.

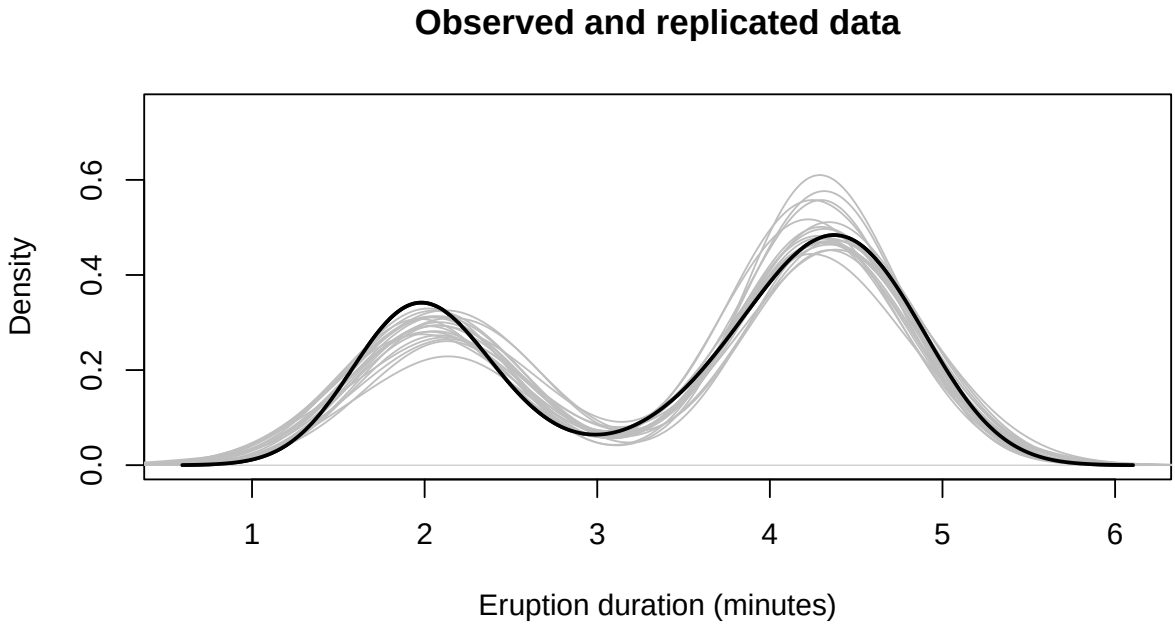
```
rep_data <- function(t) {
  z <- runif(n) < pooled$pi_low[t]
  rnorm(n, ifelse(z, pooled$mu_low[t], pooled$mu_high[t]), pooled$sigma[t])
}
skew <- function(x) mean((x - mean(x))^3) / sd(x)^3
kurt <- function(x) mean((x - mean(x))^4) / sd(x)^4 - 3
```

```r
set.seed(1)
rep_idx <- sample(nrow(pooled), 500)
stat_rep <- t(sapply(rep_idx, function(t) {
  yr <- rep_data(t)
  c(skew = skew(yr), kurt = kurt(yr))
}))
stat_obs <- c(skew = skew(y), kurt = kurt(y))

ppp <- colMeans(sweep(stat_rep, 2, stat_obs, ">="))
round(rbind(observed = stat_obs, ppp = ppp), 3)
```

```
##            skew   kurt
## observed -0.414 -1.512
## ppp       0.318  0.866
```

```r
# Density of the data against densities of a few replicated data sets
plot(density(y), lwd = 2, main = "Observed and replicated data",
     xlab = "Eruption duration (minutes)",
     ylim = c(0, 0.75))
for (t in rep_idx[1:20]) lines(density(rep_data(t)), col = "gray")
lines(density(y), lwd = 2)
```



The observed skewness (-0.41) and excess kurtosis (-1.51) are not extreme under the fitted model: the posterior predictive p-values are 0.32 and 0.87. The kurtosis of the data is in the lower part of the replicated values, so the replicated data have somewhat heavier tails. The plot of the replicated densities shows the same pattern. The two modes and the gap between them are reproduced, but the observed density is higher than all 20 replicated densities at the peak of the short eruptions, and the replicates have more mass below 1.5 minutes. These two statistics are only a rough check.

The mixture is a useful description of the shape of the distribution, but this does not show that the eruptions come from two physically distinct populations.

8. Summary of Part D

- Four chains were run on the standardized data and relabelled, since chains 2 and 4 were in the swapped labelling. $\hat{R}$ is 1.00 for all parameters.
- The lower component has mean 2.05 minutes and weight 0.36, the upper component has mean 4.30 minutes and weight 0.64, and the common standard deviation is 0.38 minutes.
- Only 2 of the 272 observations (3.07 to 3.32 minutes) have an uncertain allocation, and the posterior predictive p-values for skewness (0.32) and kurtosis (0.87) are not extreme. The mixture is a useful description of the shape of the distribution, not necessarily of two physically separate populations.